

- SETS & RELATIONS
- Collection of distinct objects
- Each set's objects are drawn from a larger domain of objects all of which have the same type i.e. homogeneous $x \in S$
- We can define the set by describing the property that element belongs to.
- $\{x:S \mid P(x)\}$
- Form new set that belongs to Domain S and satisfy the predicate P
- e.g. $\{x : \mathbb{N} \mid x < 10\}$ No. less than 10
- $\{x : \mathbb{Z} \mid (\exists y : \mathbb{Z} \mid x = 2y)\}$ Even No's
- $\{x : \mathbb{N} \mid \text{false}\}$ Empty set of natural numbers

- Cardinality -
- The Cardinality (#) of a finite set is the number of its elements
- Set Operations –
 - Union : $X \cup Y \equiv \{e \mid e \in X \text{ or } e \in Y\}$
 - Intersection – $X \cap Y \equiv \{e \mid e \in X \text{ and } e \in Y\}$
 - Difference - $X \setminus Y \equiv \{e \mid e \in X \text{ and } e \notin Y\}$
- Subsets –
 - $X \subseteq Y$ iff $(\forall e \mid e \in X \Rightarrow e \in Y)$
- Power Sets –
 - The power set of set S (denoted $\text{Pow}(S)$) is the set of all subsets of S , i.e.,
 - $\text{Pow}(S) \equiv \{e \mid e \subseteq S\}$
- Sets are disjoint if they share no elements

- In the modeling we take set and divide its members into disjoint subset it is called partitions.
- E.g.
- Odd positive integers
 $\{ x : \mathbb{Z} \mid x > 0 \wedge x \bmod 2 = 1 \}$
- The squares of integers
 $\{ x : \mathbb{N} \mid (\exists y : \mathbb{Z} \mid x = y * y) \}$
- Set has at least one element
 $(\exists x \mid x \text{ in } S)$
- Set has exactly one element
 $\exists x : S \mid (\forall y : S \mid y = x)$

- Relationships-
- useful to be able to refer to structured values .
e.g. -struct, record, object fields .
- Alloy is based on relations .
- Alloy models will be built using relations (sets of tuples).
- Product- $A \times B \equiv \{(a, b) \mid a \in A \text{ and } b \in B\}$
- Relation-
binary relation- $\text{Pow}(A \times B)$
ternary relation- $\text{Pow}(A \times B \times C)$

N-ary relations
- Common Relation Structures
- A function is a relation F

- Functions-
 injective(one-one)
 surjective (onto)
 bijective
- Relation Closure-
- The closure of a relation $r: S \times S$ (written r^+) is what you get when you keep navigating through r until you can't go any farther. $r^+ \equiv r \cup (r;r) \cup (r;r;r) \cup \dots$
- Relation Transpose – $\sim r \equiv \{(b,a) \mid (a,b) \in r\}$

- Alloy constructs-
 - Static model
 - Dynamic model
 - sets, subsets, relations with multiplicity constraints
- Atoms and Relations-
 - An atom is a primitive entity
 - Relation - structure that relates atoms (set of tuples)

Unary relations: a set of names, a set of addresses and a set of books

Name = {(N0),(N1),(N2)} ← Atoms

Addr = {(D0),(D1)} ← Atoms

Book = {(B0),(B1)} ← Tuples

A **binary relation** from names to addresses

address = {(N0,D0),(N1,D1)} ← Tuples

A **ternary relation** from books to name to addresses

addr = {(B0,N0,D0), (B0,N1,D1), (B1,N1,D2)} ← Atoms

- Alloy Specifications

Signatures and Fields

Predicates and Functions

Facts

Assertions

Commands and scopes

- Signatures

Describe the entities that you want to reason about sets defined in signatures are fixed (concept related to operations and dynamic models)

- Fields

Define relations between signatures

- Simple constraints

Multiplicities on signatures

Multiplicities on relations

- Signatures

A signature introduces a set of atoms

declaration : **sig A {}**

- A set can be introduced as an extension of
- another : **sig A1 extends A {}**

- Fields: Relations are declared as fields of signatures : **Sig A { f:e }**

- E.g. Binary Relation: **sig A { f1: A } // f1 is a subset of A x A**

Ternary Relation: **sig B { f2: A -> A } // f2 is a subset of B X A x A**

- Multiplicities:

Allow us to constrain the sizes of sets

m sig A {}

sig A {f :m e}

sig A {f: e1 m -> n e2}

- There are four multiplicities

set: any number

some: one or more

lone: zero or one

one: exactly one

Sig Name, Addr{}

Sig Book { addr : Name some -> lone Addr }

none *empty set*

univ *universal set*

iden *identity relation*

Person = { (P0), (P1), (P2) }

Name = { (N0), (N1), (N2), (N3) }

none = { }

univ = { (P0), (P1), (P2), (N0), (N1),
(N2), (N3) }

iden = { (P0, P0), (P1, P1), (P2, P2),
(N0, N0), (N1, N1), (N2, N2), (N3, N3)
}

- Set Operations

+ *union*

& *intersection*

– *difference*

in *subset*

= *equality*

-> *cross product*

Person = { (P0), (P1) }

Name = { (N0), (N1) }

Address = { (A0) }

Person -> Name =

{ (P0, N0), (P1, N0), (P1, N0),
(P1, N1) }

Person -> Name -> Address =

{ (P0, N0, A0), (P1, N0, A0), (P1,
N0, A0),
(P1, N1, A0) }

$p \cdot q$

dot is the relational join operator

Given two tuples $(p_1, \dots, p_n)$ in p and $(q_1, \dots, q_m)$ in q
where $p_n = q_1$

$p \cdot q$ contains the tuple $(p_1, \dots, p_{n-1}, q_2, \dots, q_m)$

$$\{ (N0) \} \cdot \{ (N0, D0) \} = \{ (D0) \}$$

$$\{ (N0) \} \cdot \{ (N1, D0) \} = \{ \}$$

$$\{ (N0) \} \cdot \{ (N0, D0), (N0, D1) \} = \\ \{ (D0), (D1) \}$$

$$\{ (N0), (N1) \} \cdot \{ (N0, D0), (N1, D1), (N2, D3) \} = \\ \{ (D0), (D1) \}$$

$$\{ (N0, A0) \} \cdot \{ (A0, D0) \} = \{ (N0, D0) \}$$

- Boolean operators

!	not	negation
&&	and	conjunction
	or	disjunction
=>	implies	implication
	else	alternative
<=>	iff	bi-implication

four equivalent constraints:

$F \Rightarrow G \text{ else } H$

$F \text{ implies } G \text{ else } H$

$(F \ \&\& \ G) \ || \ ((!F) \ \&\& \ H)$

$(F \ \text{and} \ G) \ \text{or} \ ((\text{not } F) \ \text{and} \ H)$

- Quantifiers

all $x: e \mid F$

all $x: e_1, y: e_2 \mid F$

all $x, y: e \mid F$

all $\text{disj } x, y: e \mid FF \text{ holds on distinct } x \text{ and } y$

all F holds for every x in e

some F holds for at least one x in e

no F holds for no x in e

lone F holds for at most one x in e

one F holds for exactly one x in e

Operations on relation

~ transpose

^ transitive closure

* reflexive transitive closure

these apply only to binary relations

$$^r = r + r.r + r.r.r + \dots$$

$$*r = \text{iden} + ^r$$

$$w = \{(M0, W1), (M1, W2)\}$$

$$\sim w = \{(W1, M0), (W2, M1)\}$$

Books:

Software Abstractions: Logic Language &
Analysis

Link: ***alloy***.mit.edu/